Лабораторная работа №1. Базовый пайплайн обработки изображения

Дата выполнения: 6 ноября 2026 г.

1. Цель и задачи

Цель работы — освоить базовый пайплайн обработки изображения и научиться обосновывать выбор метода и его параметров. В работе реализованы все три варианта задания:

1. шумоподавление и повышение контраста;
2. выделение границ и контуров объектов;
3. сегментация изображения по цвету.

Для каждого варианта параметры вынесены в JSON-конфиг, программа запускается из командной строки, а результаты сохраняются в виде изображений «до/после», масок, графиков и CSV-таблиц с количественными метриками.

2. Инструменты и данные

Использованы Python 3.10+ (проверено на Python 3.14), NumPy, OpenCV, Pillow, scikit-image и matplotlib. Встроенные изображения scikit-image (`camera`, `coins`, `astronaut`) закэшированы в `data/skimage/`, поэтому повторные запуски выполняются офлайн и воспроизводимо.

Вариант	Изображение	Размер	Тип
1	`data.camera`	512 × 512	полутоновое
2	`data.coins`	303 × 384	полутоновое
3	`data.astronaut`	512 × 512	цветное RGB

3. Воспроизводимость

Единая точка входа:

```
python -m src all --config configs/default.json --out out
```

Отдельные варианты запускаются командами `python -m src 1`, `python -m src 2` и `python -m src 3`. Любой параметр конфига можно переопределить без правки кода:

```
python -m src 1 --set variant1.gaussian_sigma=1.6 \
                --set variant1.clahe_clip_limit=2.5
```

Для синтетического шума используется фиксированное зерно генератора, что делает графики и таблицы воспроизводимыми. Все численные результаты дублируются в CSV-файлах в папках `out/variant1` – `out/variant3`.

4. Вариант 1. Шумоподавление и контраст

4.1. Методика

К исходному изображению `camera` добавлялся синтетический шум двух типов:

- аддитивный гауссов шум со СКО σ = 5, 10, 15, 20, 30 и 40 уровней яркости;
- импульсный шум «соль и перец» с долей повреждённых пикселей 1, 2, 5, 8 и 12 %.

Для каждого уровня выполнялось по три независимые реализации шума. Одна и та же зашумлённая реализация обрабатывалась двумя фильтрами:

- гауссовым фильтром 5 × 5 с σ = 1.4;
- медианным фильтром 5 × 5.

Восстановление сравнивалось с чистым изображением по PSNR и SSIM (`data_range = 255`). Результаты усреднялись по реализациям, на графиках показана полоса ± одно стандартное отклонение.

4.2. Результаты по шуму

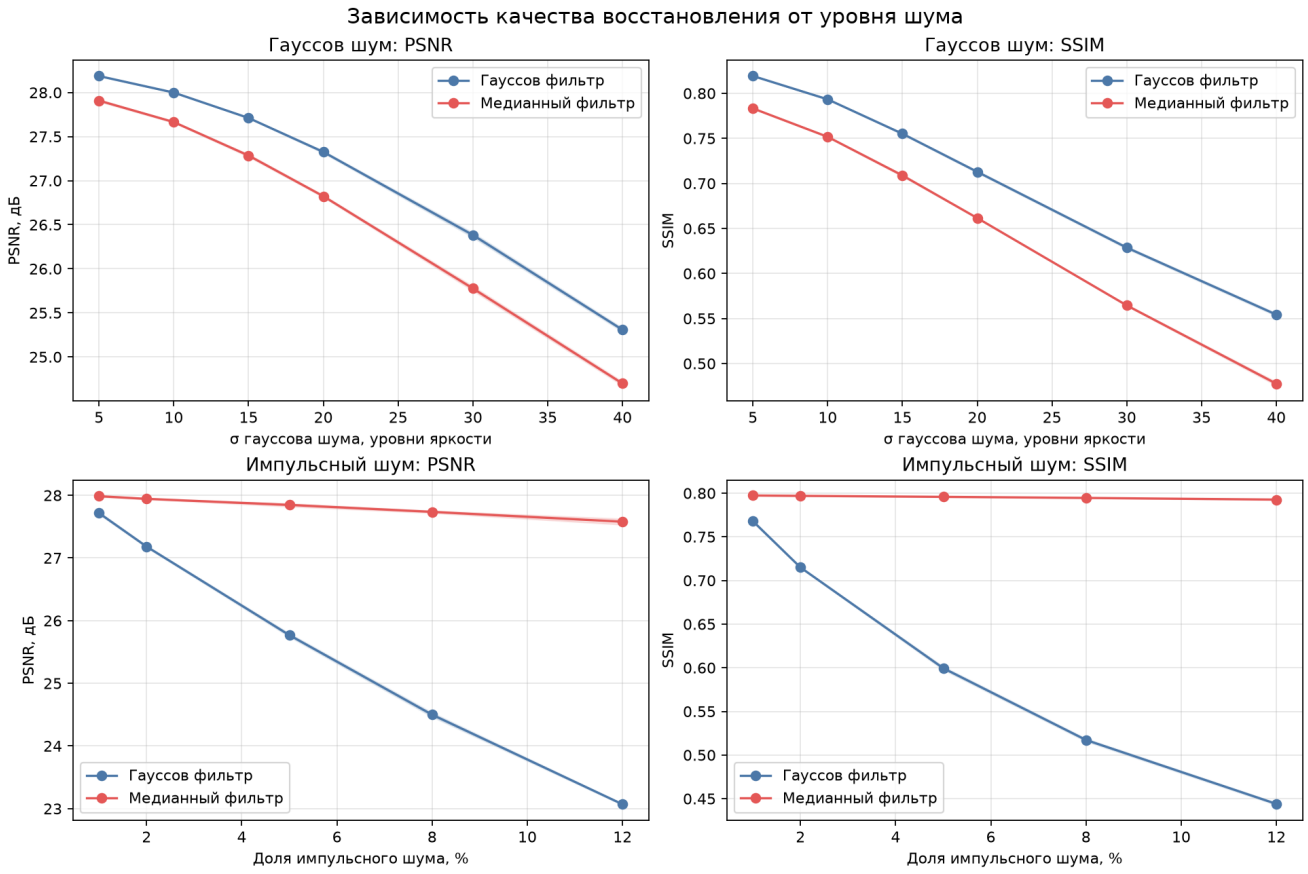


Рис. 1. Зависимость PSNR/SSIM от уровня гауссова и импульсного шума.

Выборочные значения:

σ гауссова шума	Гауссов фильтр, PSNR	Гауссов фильтр, SSIM	Медианный фильтр, PSNR	Медианный фильтр, SSIM
10	28.00	0.793	27.66	0.752
20	27.33	0.713	26.82	0.662
30	26.38	0.629	25.78	0.565
40	25.31	0.554	24.70	0.478

Доля импульсного шума	Гауссов фильтр, PSNR	Гауссов фильтр, SSIM	Медианный фильтр, PSNR	Медианный фильтр, SSIM
2 %	27.18	0.715	27.94	0.797
5 %	25.77	0.599	27.85	0.796
8 %	24.50	0.517	27.74	0.795
12 %	23.07	0.444	27.58	0.793

На гауссовом шуме гауссов фильтр устойчиво лучше: при σ = 40 он даёт PSNR 25.31 дБ против 24.70 дБ и SSIM 0.554 против 0.478. Это ожидаемо: линейное усреднение оптимально подавляет аддитивный нормальный шум.

На импульсном шуме картина обратная. Медианный фильтр почти не деградирует при росте доли выбросов: PSNR меняется с 27.94 до 27.58 дБ, SSIM остаётся около 0.79. Гауссов фильтр «размазывает» выбросы по окрестности, и при 12 % шума PSNR падает до 23.07 дБ, SSIM — до 0.444. Даже при 2 % импульсов медиана выигрывает, поскольку одиночные экстремальные пиксели сильно влияют на линейное среднее.

4.3. Выбор параметров фильтров

Дополнительно выполнено сканирование параметров при фиксированном гауссовом шуме σ = 25:

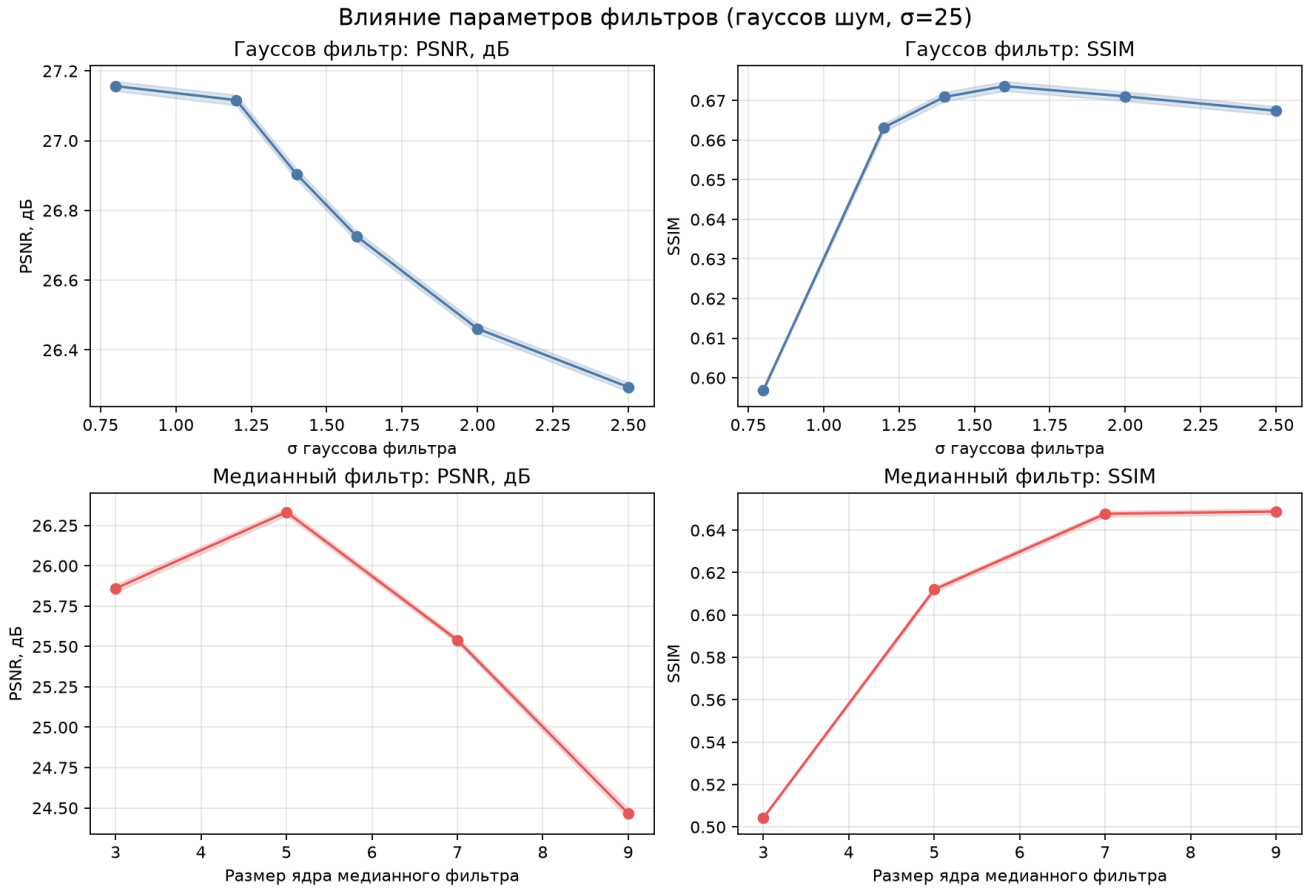


Рис. 2. Чувствительность PSNR/SSIM к σ гауссова фильтра и размеру ядра медианного фильтра.

Фильтр	Параметр	PSNR, дБ	SSIM
Гауссов	σ = 1.2	27.12	0.663
Гауссов	σ = 1.4	26.90	0.671
Гауссов	σ = 1.6	26.73	0.674
Гауссов	σ = 2.0	26.46	0.671
Медианный	ядро 3	25.86	0.504
Медианный	ядро 5	26.33	0.612
Медианный	ядро 7	25.54	0.648
Медианный	ядро 9	24.47	0.649

У гауссова фильтра PSNR максимален при малом σ = 0.8, но SSIM при этом всего 0.597: фильтр почти не подавляет шум и сохраняет высокочастотные детали формально, но изображение остаётся шумным. Максимум SSIM достигается при σ ≈ 1.6, а PSNR падает плавно. В качестве компромисса выбрано σ = 1.4: SSIM 0.671 близок к максимуму 0.674, а PSNR 26.90 дБ выше, чем при σ = 1.6.

Для медианного фильтра ядро 5 × 5 даёт лучший PSNR (26.33 дБ) при разумном SSIM 0.612. Ядра 7 и 9 немного повышают SSIM, но заметно снижают PSNR (до 24.47 дБ) из-за чрезмерного сглаживания. Поэтому выбрано ядро 5 × 5.

4.4. Сравнение выравнивания гистограммы и CLAHE

Для исходного изображения сравнивались глобальное выравнивание гистограммы (`cv2.equalizeHist`) и адаптивное выравнивание CLAHE (clip limit 2.0, плитка 8 × 8). Поскольку оба метода меняют яркости пикселей, PSNR/SSIM считались относительно исходного изображения; дополнительно оценивались средняя яркость, глобальное СКО и энтропия.



Рис. 3. Исходное изображение, глобальное выравнивание и CLAHE.

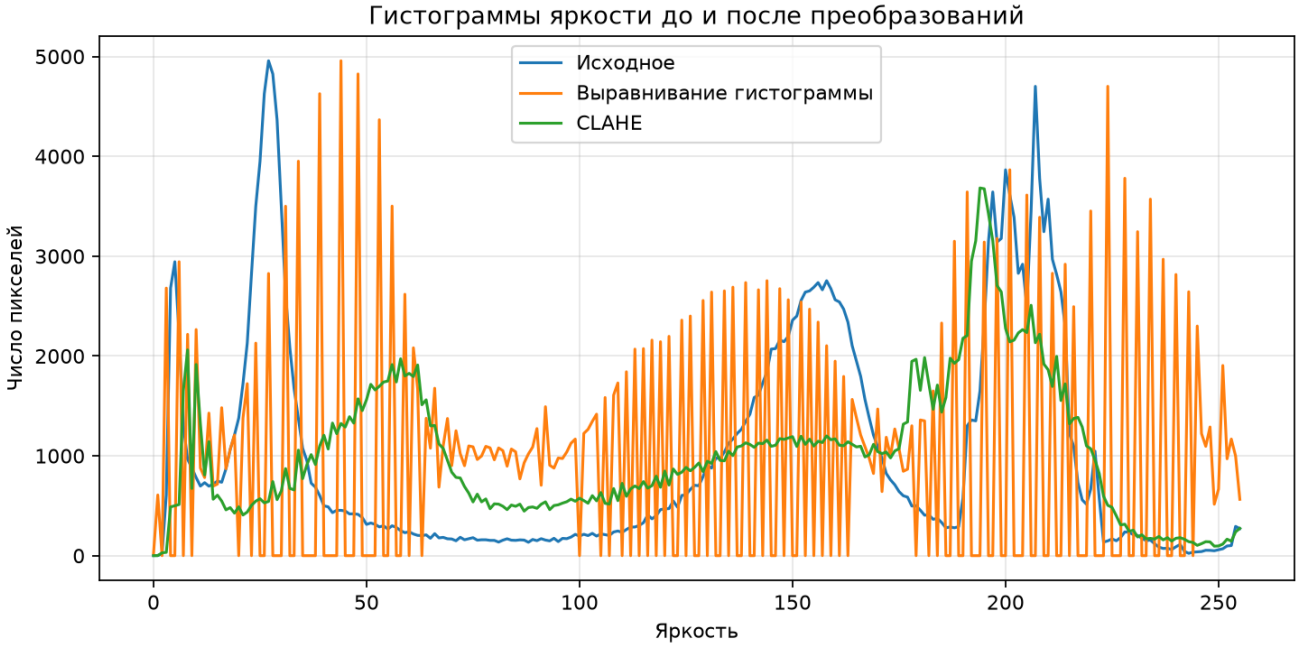


Рис. 4. Гистограммы яркости до и после преобразований.

Метод	PSNR к исходному	SSIM к исходному	Средняя яркость	СКО	Энтропия
Без обработки	—	1.000	129.06	73.64	7.232
Выравнивание гистограммы	22.03	0.859	128.60	73.67	6.945
CLAHE	22.11	0.834	132.75	67.64	7.687

Глобальное выравнивание почти не меняет среднюю яркость и глобальное СКО (73.64 → 73.67), но делает гистограмму «гребенчатой»: из-за дискретности многие уровни яркости не используются, поэтому энтропия даже снижается (7.232 → 6.945). Визуально изображение становится более жёстким, часть светлых участков уходит в пересвет.

CLAHE повышает энтропию до 7.687 и среднюю яркость до 132.75, при этом глобальное СКО снижается до 67.64. Это не противоречие: CLAHE усиливает локальные перепады в пределах плиток, а не растягивает весь диапазон глобально. Для естественных изображений такой вариант обычно предпочтительнее: он не «выжигает» света и сохраняет локальную детализацию. При этом SSIM у CLAHE ниже (0.834 против 0.859), потому что локальные преобразования сильнее меняют пиксели относительно оригинала. PSNR и SSIM здесь нельзя использовать как единственный критерий: они штрафуют любое изменение тона, даже если контраст субъективно улучшился. Поэтому вывод делается по совокупности метрик и визуального сравнения.

4.5. Совместный пайплайн и остаточный шум



Рис. 5. Фильтрация зашумлённого изображения и последующее применение CLAHE.

Этап	PSNR к чистому	SSIM к чистому
Зашумлённое изображение (σ = 25)	20.61	0.302
Гауссов фильтр 5 × 5, σ = 1.4	26.90	0.671
Гауссов фильтр + CLAHE	19.94	0.435

После фильтрации остаётся шумовая составляющая, которую CLAHE усиливает вместе с полезными локальными контрастами. В результате PSNR падает ниже уровня зашумлённого изображения, а SSIM хотя и растёт, но визуально картинка становится зернистой. Практический вывод: при заметном шуме нельзя ставить CLAHE сразу после слабого фильтра; нужно либо сильнее подавить шум, либо уменьшить clip limit, либо применять контрастирование до добавления шума (если это возможно в реальном пайплайне).

4.6. Вывод по варианту 1

- Для гауссова шума выбирается гауссов фильтр, для импульсного — медианный.
- Для гауссова фильтра 5 × 5 компромиссное σ ≈ 1.4; для медианного — ядро 5 × 5.
- CLAHE предпочтительнее глобального выравнивания для естественных изображений: выше энтропия, мягче тональные изменения.
- PSNR/SSIM полезны для сравнения фильтров, но для оценки контрастных преобразований их нужно дополнять метриками контраста и визуальным анализом.
- Совместный пайплайн «фильтр → CLAHE» усиливает остаточный шум; это ограничение важно учитывать при выборе силы фильтра.

5. Вариант 2. Границы и контуры объектов

5.1. Методика

Использовалось изображение `coins` (303 × 384), содержащее 24 монеты на контрастном фоне. Перед выделением границ применялось лёгкое гауссово размытие с σ = 1.0 для подавления мелкой текстуры.

Сравнивались три метода:

- **Sobel** — модуль градиента, порог по 85-му перцентилю;
- **Scharr** — аналогично, с более точными ядрами для поворотов;
- **Canny** — гистерезисный детектор с порогами 60/150.

Маски очищались морфологическим закрытием эллиптическим ядром 3 × 3 (одна итерация) и удалением связных компонент площадью менее 20 пикселей. Затем искались внешние контуры (`RETR_EXTERNAL`), которые фильтровались по площади: от 500 до 5000 пикселей. Для каждого метода выводилось число объектов на кадре, площади, периметры и компактность.

5.2. Результаты

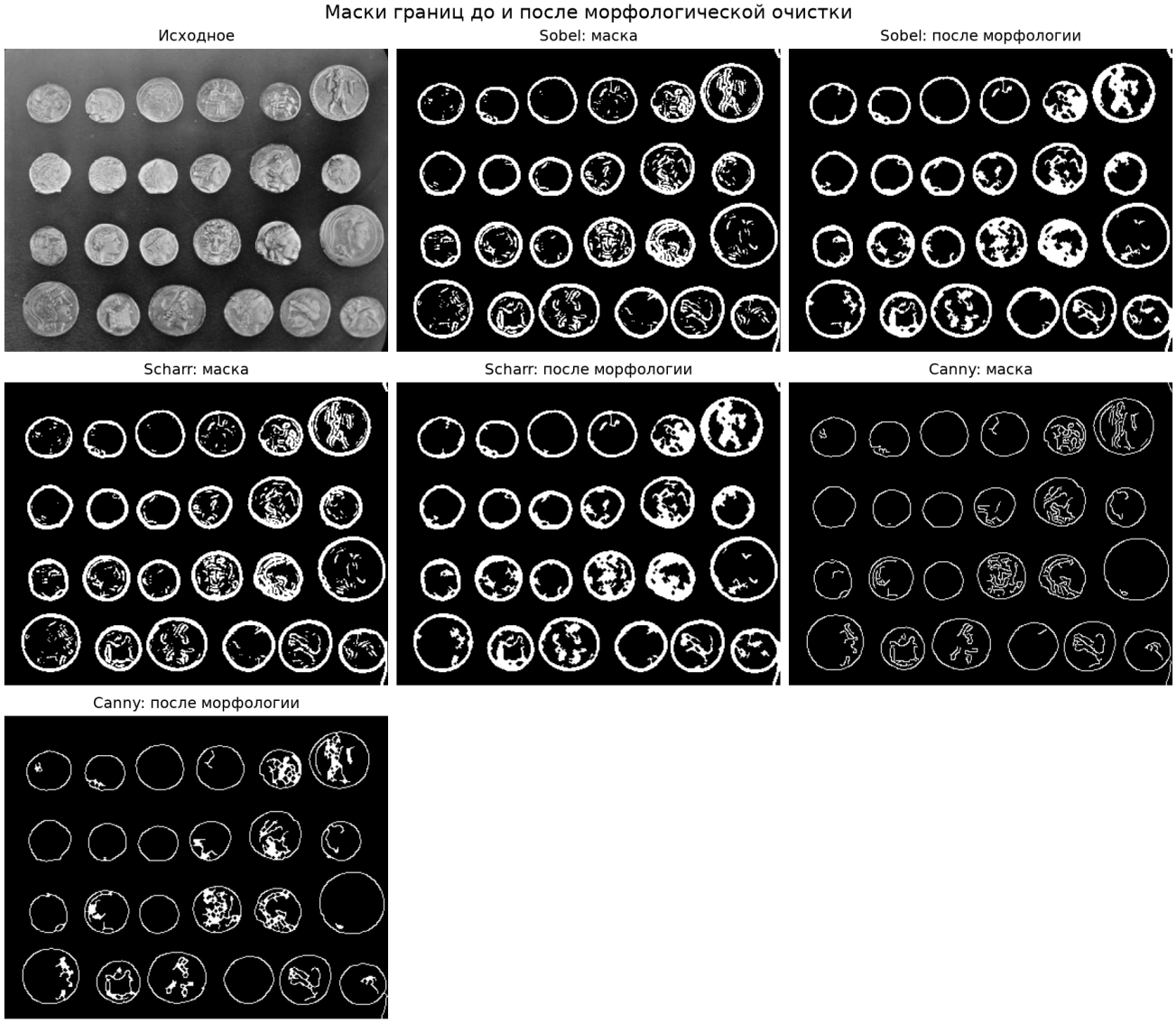


Рис. 6. Маски Sobel, Scharr и Canny до и после морфологической очистки.

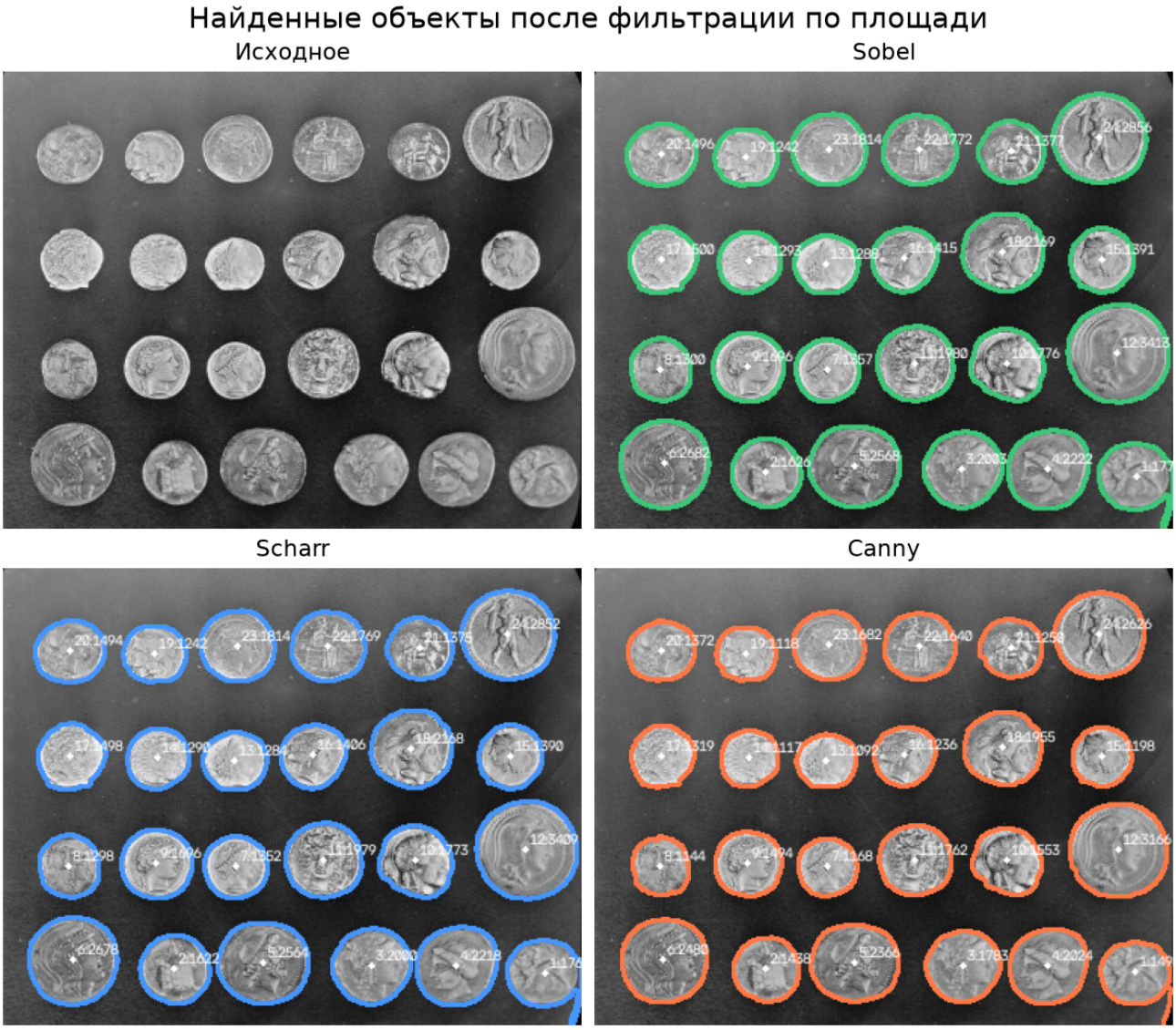


Рис. 7. Контуры, прошедшие фильтрацию по площади.

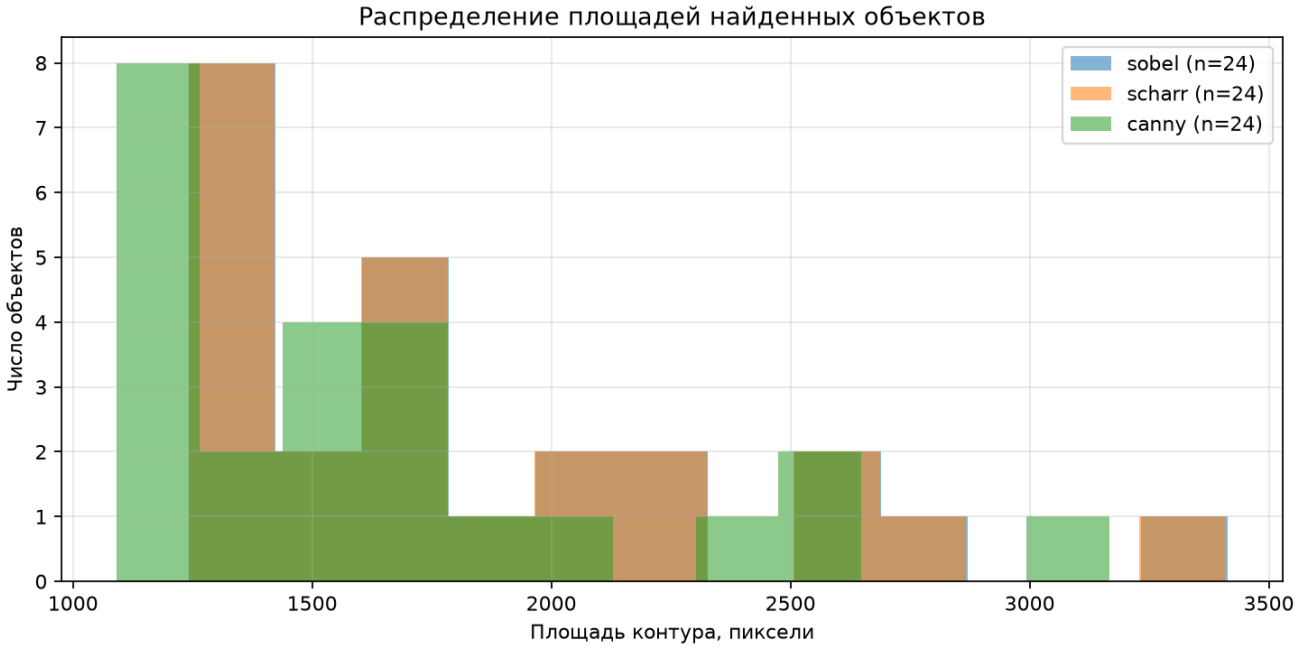


Рис. 8. Распределение площадей найденных контуров.

Метод	Найдено объектов	Средняя площадь, px	Минимум, px	Максимум, px
Sobel	24	1833.7	1242.5	3413.0
Scharr	24	1830.4	1241.5	3409.0
Canny	24	1645.0	1092.0	3166.5

Все три метода после очистки и фильтрации нашли ровно 24 объекта, то есть столько же, сколько монет на изображении. Ложных срабатываний на фоне нет: мелкие компоненты удалены порогом по площади, а крупные контуры соответствуют монетам.

Средняя площадь у Canny меньше (1645 px), чем у Sobel/Scharr (около 1832 px). Canny строит тонкую центральную линию границы, поэтому внешний контур проходит чуть внутри монеты. Sobel и Scharr дают более толстый отклик, и внешняя граница оказывается немного снаружи. Разница в 10–12 % по площади — методическая, а не ошибка детекции.

Порог 85-го перцентиля выбран в ходе подбора: при 90-м перцентиле Sobel и Scharr находили только 20 монет из 24 (у части монет граница оказывалась разомкнутой). Снижение порога до 80–85 % замыкает контуры, но не добавляет крупных ложных объектов. Увеличение морфологического ядра до 5–7, наоборот, снижает число находок: близкие фрагменты границ объединяются, а часть контуров теряется. Поэтому использовано ядро 3 × 3.

5.3. Вывод по варианту 2

- Sobel, Scharr и Canny при правильном подборе порогов дают одинаковое количество объектов на данном кадре.
- Canny удобнее для последующего поиска контуров: меньше внутренней текстуры и тоньше границы.
- Sobel/Scharr чувствительнее к слабым перепадам, но требуют более аккуратной морфологической очистки.
- Фильтрация по площади — эффективный способ убрать шумовые компоненты; результат зависит от порога и размера морфологического ядра.

6. Вариант 3. Сегментация по цвету

6.1. Методика

Изображение `astronaut` переводилось из RGB в HSV. В HSV цвет (тон) отделён от яркости, что делает пороги устойчивее к освещению. Заданы три класса:

Класс	H (OpenCV, 0–179)	S	V	Мин. площадь
Оранжевый костюм	2–12	130–255	80–255	1000
Кожа (лицо)	8–18	55–130	80–255	1000
Синие объекты	90–140	70–255	25–255	150

Диапазоны подобраны по распределению HSV: оранжевый костюм имеет H ≈ 6–10 и высокую насыщенность, кожа — H ≈ 10–16 при S ≈ 60–100, синие элементы (флаг, нашивки, блики) — H ≈ 100–130. Классы сделаны взаимоисключающими: при пересечении приоритет имеет класс, указанный в конфиге раньше (оранжевый → кожа → синий).

Маски очищались морфологическим закрытием и открытием эллиптическим ядром 9 × 9, затем удалялись связные компоненты площадью менее 25 пикселей. Объекты и их площади вычислялись через `skimage.measure.regionprops`; дополнительно фильтровались объекты меньше минимальной площади класса.

6.2. Результаты

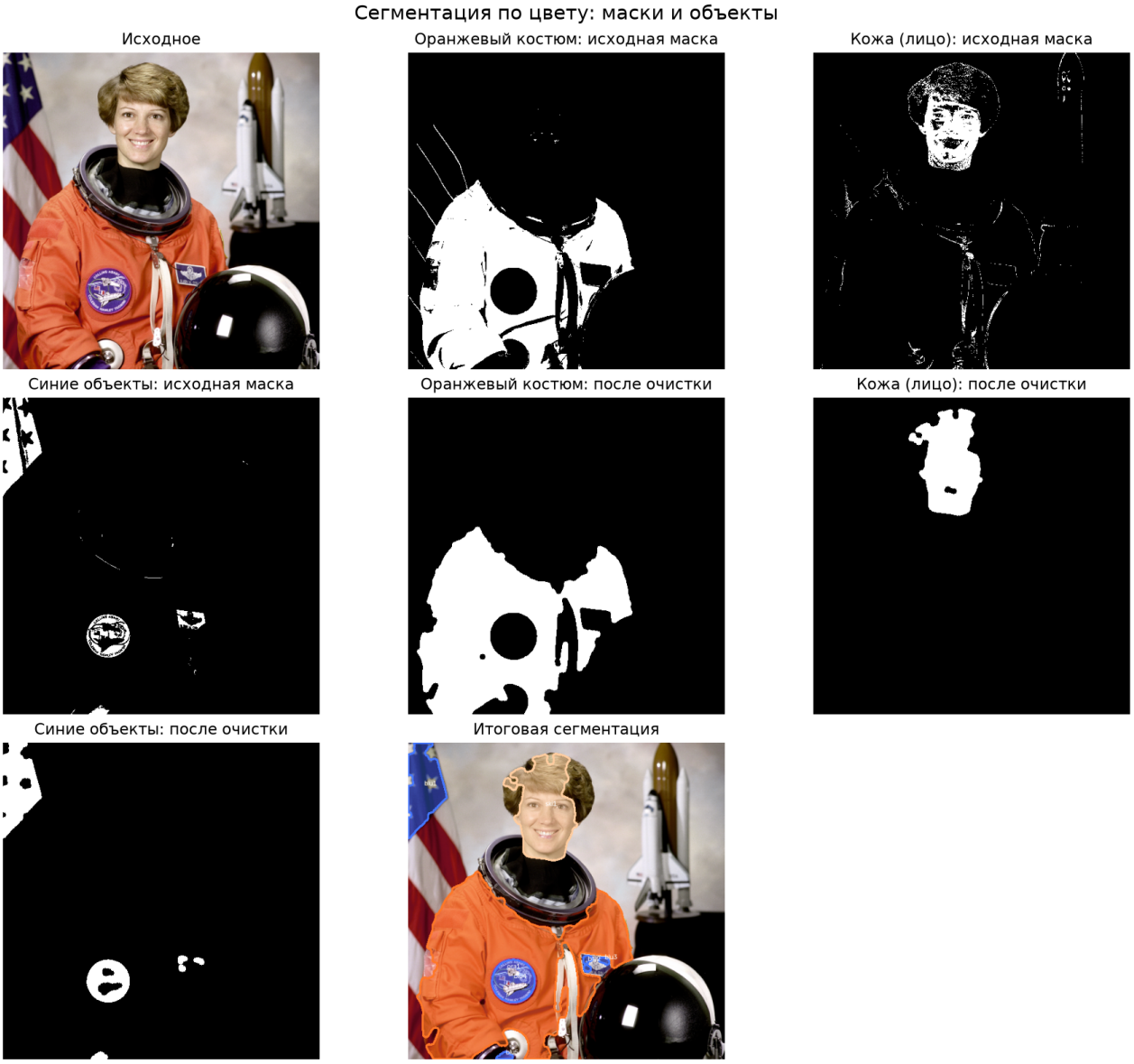


Рис. 9. Исходное изображение, маски классов до/после очистки и итоговая разметка.

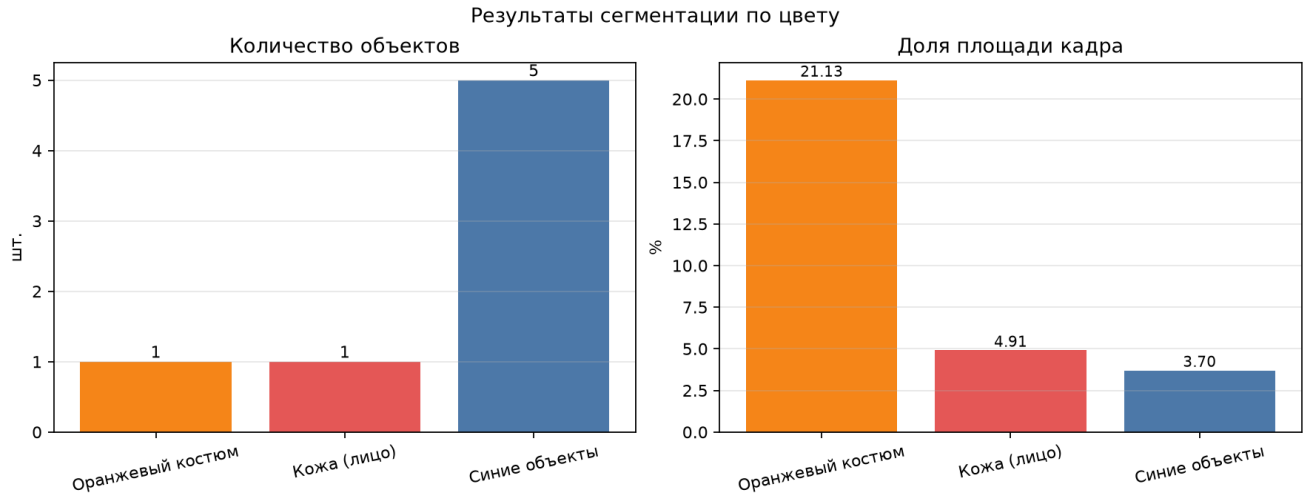


Рис. 10. Количество объектов и доля площади кадра по классам.

Класс	Объектов	Суммарная площадь, px	Доля кадра	Средняя площадь, px
Оранжевый костюм	1	55 400	21.13 %	55 400
Кожа (лицо)	1	12 866	4.91 %	12 866
Синие объекты	5	9 706	3.70 %	1 941.2
Итого	**7**	**77 972**	**29.74 %**	—

Синие объекты — это пять связных областей: синее поле флага (5893 px), нашивка на левой стороне груди (3120 px), два фрагмента правой нашивки (287 и 157 px) и небольшой блик на шлеме (249 px). Фрагментация нашивки связана с белой вышивкой и бликами внутри неё: `regionprops` считает связные компоненты, а не семантические объекты.

Ядро 9 × 9 выбрано экспериментально. При 5 × 5 костюм распадался на две крупные области (торс и рукав). При 7 × 7 костюм уже был единым, но маска кожи давала второй объект площадью 743 px — светлый блик у границы волос; его отсекает минимальная площадь 1000 px. Ядро 9 × 9 дополнительно объединяет часть мелких фрагментов синих нашивок. Для наших задач это приемлемо; на изображениях с близко расположенными объектами такое ядро может их слиять.

6.3. Вывод по варианту 3

- HSV-пороги позволяют надёжно выделить три цветовых класса на изображении `astronaut`.
- Морфологическая очистка и фильтрация по площади убирают шум, но могут дробить объекты со сложной текстурой (нашивки, блики).
- `regionprops` даёт площадь, периметр, центроид, bbox и компактность; эти данные сохранены в `out/variant3/regions.csv`.
- Пороги подобраны для конкретной сцены; для произвольных фотографий требуется калибровка или интерактивный подбор.

7. Общие выводы

1. Выбор метода определяется типом искажения или объекта, а не универсальным «лучшим» алгоритмом: гауссов фильтр лучше для гауссова шума, медианный — для импульсного; Canny удобнее для контуров, Sobel/Scharr — для анализа градиентов.
2. Параметры нужно подбирать количественно. Сканирование σ и размера ядра показало компромисс между подавлением шума и сохранением деталей.
3. PSNR и SSIM полезны для сравнения восстановления с эталоном, но плохо подходят как единственный критерий повышения контраста: любое тональное преобразование снижает эти метрики. Для контраста нужны энтропия, локальный контраст и визуальная оценка.
4. Морфология — мощный, но не нейтральный инструмент: она закрывает разрывы и удаляет шум, однако может объединять или дробить объекты.
5. Все результаты воспроизводимы: параметры вынесены в конфиг, зерно генератора шума фиксировано, CLI и README содержат команды запуска.

8. Ограничения

- Синтетический шум не полностью моделирует реальные сенсорные искажения (коррелированный шум, артефакты сжатия, неравномерное освещение).
- Пороги Sobel/Scharr/Canny и HSV подобраны для конкретных тестовых изображений; на других сценах их нужно адаптировать.
- PSNR и SSIM чувствительны к сдвигам и тональным изменениям; для оценки субъективного качества контраста использовались дополнительные метрики.
- `regionprops` считает связные компоненты, поэтому текстурированные объекты могут распадаться на несколько областей.

9. Приложение. Команды и файлы

```
# установка
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# все варианты
python -m src all

# переопределение параметров
python -m src 2 --set variant2.sobel_percentile=85 \
                --set variant2.min_area=500
python -m src 3 --set variant3.morph_kernel=9

# сборка PDF
pip install -r requirements-report.txt
python scripts/build_report.py
```

Основные результаты: `out/variant1`, `out/variant2`, `out/variant3`. Полные таблицы метрик — в CSV-файлах соответствующих папок, сводки — в `summary.json`. Исходный код разделён на общие утилиты (`src/common.py`) и модули вариантов (`src/variant1.py`, `src/variant2.py`, `src/variant3.py`).